four elements. The values for each array are separated by commas and bracketed with curly braces like normal arrays, but commas also separate each individual array and the whole thing has a pair of curly braces. In this way `arrayTwo[0][0] == 1`, `arrayTwo[0][1] == 2`, `arrayTwo[1][0] == 3`, and `arrayTwo[1][1] == 4` are all true. You can think of a two-dimensional array as a tiny spreadsheet with rows and columns of data. We can even rewrite this array declaration as follows:

```
int arrayTwo[2][2] = {
  {1, 2},
  {3, 4}
};
```

Because white space doesn't count, we can use it to lay our code out in a way that makes sense to us. In this case we put the values together in a little table. Building on this idea, two-dimensional arrays are particularly useful on the Arduino for creating patterns or animations using a row or grid of LEDs. Imagine that we have 6 LEDs each individually connected to pins 2–7 and ground. We could set up an array to declare the pin numbers like so:

```
int ledPins[] = {2, 3, 4, 5, 6, 7};
```

And then we could use a multidimensional array to create an animation pattern:

```
int pattern[3][6] = {
  {0,0,1,1,0,0},
  {0,1,0,0,1,0},
  {1,0,0,0,0,1}
};
```

Ones are used to represent on or `HIGH` and zeros are off or `LOW`. This pattern could then be used to create an animation fitting for a Cylon with LEDs that light up in the middle and expand to the outside LEDs. To make it work, we would use something like the following code fragment:

```
for (int x=0; x<3; x++) {
  for (int y=0; y<6; y++) {
    digitalWrite(ledPins[y],  pattern[x][y]);
  }
  delay(250);
}
```

Here we have two nested `for` loops, the first is used to increment through the three possible animations using the `x` counter. The second for loop increments the `y` counter six times and uses that counter to both increment each element in the `ledPin[]` array, as well as each of the six elements in the animation arrays. This way when we use the `digitalWrite()` function, `ledPins[y]` will set each of the six pins to whatever state is in `pattern[x][y]`. So beginning with the first loop, `x == 0`, then the second loop begins with `y == 0`, so the led at index 0 or pin 2 is set to the condition of pattern at index 0,0, which is 0 or `LOW`. The counter `y` is incremented so that `y == 1`, so the LED at index 1 or pin 3 is set to the condition of pattern at index 0, 1, which is also `LOW`. `y` is incremented again, turning pin 4 to the condition at `pattern[0][2]`, which in this case is 1 or `HIGH`. This repeats until each of the six LEDS has been turned on or off according to the first pattern. All of this happens in rapid succession and if it weren't for the short delay to slow things down when the `y` loop has completed, we would only see the briefest flicker. Once the delay is over, the `x` loop is incremented and the `y` loop begins again with a new pattern.

This is only the briefest introduction to multidimensional arrays. We could use three-dimensional arrays to create a whole set of animations for an 8×8 LED matrix or to store the X, Y, and Z readings from an accelerometer. Maybe we could display an entire series of information on an LCD for things